

LIDAR

El subsistema LIDAR fue incorporado al autopiloto de la embarcación con el propósito de implementar una función de seguridad por proximidad frontal. Su función principal es detectar la distancia a un obstáculo ubicado en la dirección de apuntamiento del sensor y, a partir de esa información, generar dos acciones:

1. una advertencia acústica progresiva mediante buzzer;
2. una orden de detención cuando la distancia medida desciende por debajo de un umbral mínimo de seguridad.

La lógica fue diseñada para operar de manera autónoma y configurable desde la estación remota mediante telemetría, permitiendo modificar los límites de seguridad sin recompilar el firmware.

es del tipo **LIDAR-Lite V3HP GARMIN**

trabaja a **905 nM**

la comunicación con el micro es a través de I2C1 a una velocidad de 100Khz establecida por la variable,

```
#define I2C1_TIMINGR_VALUE  
((uint32_t)0x70B03839U)
```

la máxima velocidad tolerable es de 400Khz (Fast Mode)

el sensor necesita 22 mseg para su arranque



registros más importantes que utiliza el sensor,

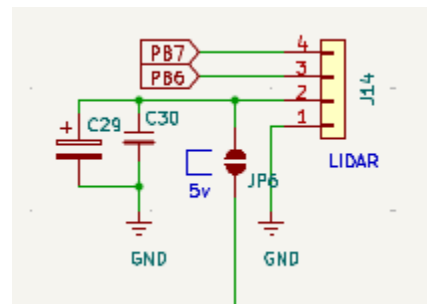
- 0x00 ACQ_COMMAND: arranca una medición al escribir un valor no nulo. También puede hacer hard reset por el bit 0, pero solo si antes habilitaste 0x06 LEGACY_RESET_EN.
- 0x01 STATUS: es el registro clave de estado. Bit 5 = salud, bit 4 = regulación DC, bit 3 = pico detectado, bit 2 = overflow de referencia, bit 1 = overflow de señal, bit 0 = busy.
- 0x02 SIG_COUNT_VAL: fija la cantidad máxima de adquisiciones; Garmin indica que esto gobierna el compromiso entre tasa de medición y alcance máximo. Valor inicial 0xFF.
- 0x04 ACQ_CONFIG_REG: controla el modo de adquisición. Su valor inicial es 0x08, y el bit 3 permite habilitar o deshabilitar quick termination.
- 0x0E SIGNAL_STRENGTH: te da una medida útil de la intensidad del retorno.
- 0x0F / 0x10 FULL_DELAY_HIGH/LOW: contienen la distancia en centímetros.
- 0x1C THRESHOLD_BYPASS: permite forzar un umbral simple de detección; Garmin dice que 0x20-0x60 suele funcionar bien si querés salir del algoritmo por defecto.

- 0x48 HEALTH_STATUS: útil al arranque para diagnóstico de hardware. En tu prueba, el 0x1F fue coherente con un sensor sano.
- 0x65 POWER_CONTROL: permite deshabilitar el receptor para ahorrar unos 40 mA cuando no estás midiendo.

Estructura de datos obtenida,

```
typedef struct
{
    uint16_t distance_cm;
    uint8_t status;
    uint8_t signal_strength;
    uint8_t health_status;
} LIDAR_Measurement_t;
```

El sensor está conectado a J14 de la placa madre y controlado con los puertos PB6 y PB7 . Para economizar energía , lo que voy a utilizar es el control a través del registro POWER CONTROL , el cual se encarga de apagar el receptor cuando el sensor esta inactivo.



Este switching de alimentación del receptor vale la pena si la periodicidad de mediciones con el LIDAR es superior a 100 msecs. Mientras mayor sea el tiempo de medición mejor es el rendimiento que vamos a obtener con este método. Podemos obtener un ahorro de 40mA cada intervalo de medición.

El sistema consume aproximadamente 65mA durante el Idle y 85mA durante la adquisición.

Como aplicación le vamos a dar una funcionalidad de aviso de proximidad de obstáculo asociado con el buzzer. Por configuración de interfaz le cargaremos la distancia mínima tolerable, distancia en la cual el vehículo deja de avanzar por seguridad. Y además cargaremos una distancia de advertencia, a partir de la cual, a medida que vamos reduciéndola la frecuencia de aviso del buzzer será mayor. Ese aviso similar se enviará por telemetría para que desde la interfaz tengamos un aviso visual también y porque no sonoro de la misma forma.

Para la evaluación de la distancia no utilizo distancia cruda, ya que el LIDAR suele tener pequeñas variaciones. En lugar de eso utilizo distancia filtrada, es decir, hace un promedio de entre 3 a 5 muestras para determinar la distancia actual. Es decir,

$$d_{\text{filtrada}} = d_{\text{filtrada}} + \alpha * (d_{\text{nueva}} - d_{\text{filtrada}});$$

siendo alpha aprox 0,2 f y de esta forma evito que el buzzer y la detención titilen.

Voy a evaluar el valor de x para saber si está en zona de alerta y si llegó a la distancia mínima, y eso lo vamos a relacionar con un período de beep para el buzzer.

$$x = (\text{DIST_MAX_CM} - \text{distancia}) / (\text{DIST_MAX_CM} - \text{DIST_MIN_CM})$$

$$\text{beep_period_ms} = 1000 - x * 900$$

Tendré también una `DIST_MIN_RELEASE_CM` que será un porcentaje de la `DIST_MIN_CM` que será una distancia de recuperación, es decir, si llega a la mínima, luego para volver al estado anterior tiene que superar esa `DIST_MIN_RELEASE_CM`. Es una especie de histéresis para evitar transitorios.

Entonces el período de beeps será cerca de `DIST_MAX_CM` de aprox 1000 ms , y cerca de `DIST_MIN_CM` de aprox 100 ms. Y al llegar a la distancia mínima, buzzer será continuo y se detendrá el avance.

Hay valores por defecto de estas distancias de seguridad y distancia crítica, actualmente está en 40 cm. como distancia crítica y 2 metros como distancia de seguridad, las distancias release de cada una se calcula con 10 % de su valor. Recordemos que estos valores son los que tiene que superar para volver al estado anterior. Es decir, si mi distancia de seguridad es de 2 metros y comienza a sonar el buzzer, para que deje de sonar el buzzer tiene que superar en 10% esa distancia de seguridad que sería cuando la distancia es superior a 2,2 metros.

En el panel de control de la Interface de la PC, podemos acomodar estos valores de distancia manualmente, colocando tanto la distancia de seguridad como la distancia crítica deseada según el móvil en el que se va a utilizar.

También tenemos la posibilidad de cortar la alarma del buzzer por si por comodidad quisiéramos que este en silencio.

En todos los casos, se envía por telemetría estos nuevos valores al microcontrolador una vez que presionamos el boton de “Enviar Distancias” .

Las distancias se seleccionan en metros.

Validación de la medición

No toda lectura obtenida del sensor se considera automáticamente válida. Antes de actualizar la lógica de proximidad se verifica:

- que la medición haya sido completada correctamente;
- que exista detección de pico válida;
- que la intensidad de señal supere un mínimo;
- que el estado del sensor indique operación saludable;
- que la distancia no sea nula.

En caso de medición inválida, la lógica puede optar por conservar el último estado útil y evitar una decisión espuria.

Lógica de la alarma acústica

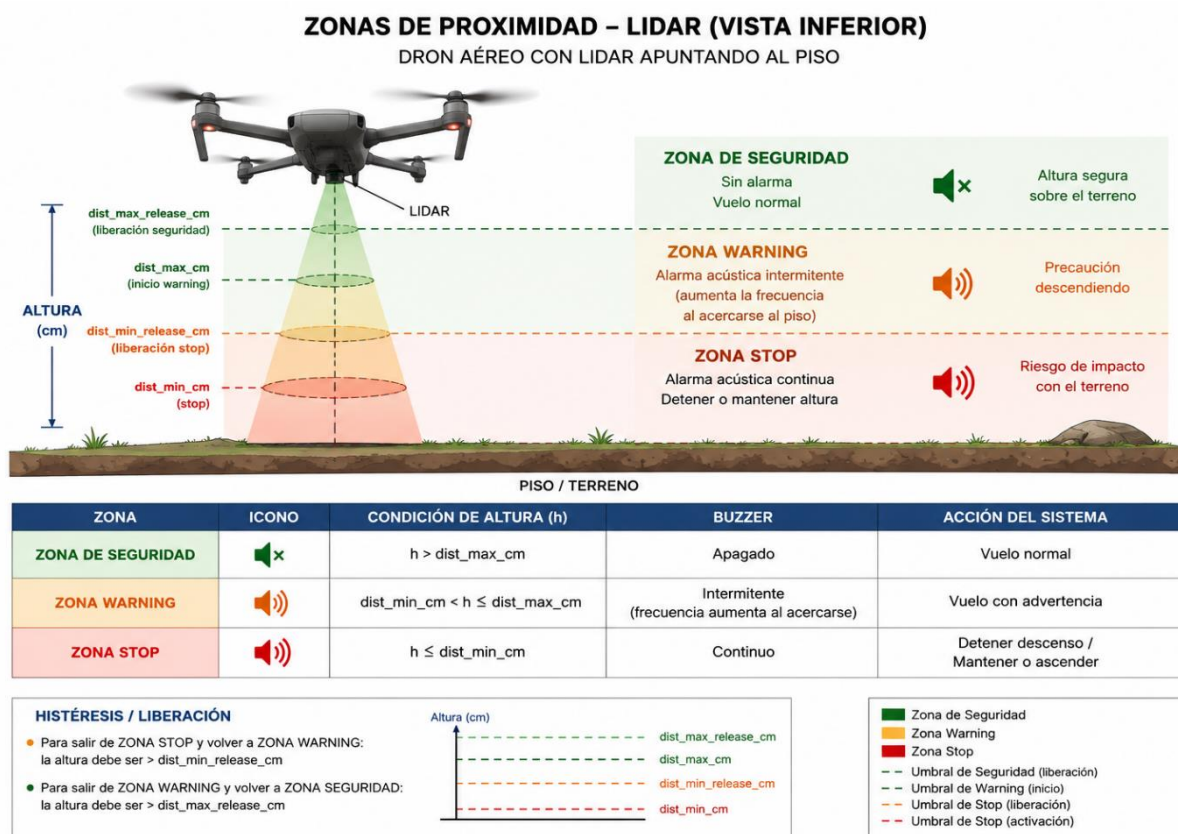
La alarma acústica se implementa a partir de dos parámetros:

- período total de repetición del beep;
- duración del tiempo activo del beep.

En la zona WARNING, la cadencia se calcula en función de la distancia filtrada. A menor distancia, menor período y, por lo tanto, mayor frecuencia de aviso.

En la zona STOP, el buzzer pasa a estado continuo.

La salida acústica no es generada directamente desde el driver del LIDAR. El módulo de proximidad solo calcula el estado lógico del buzzer. La activación física del pin PC6 se realiza desde el programa principal.



El subsistema LIDAR no actúa directamente sobre el motor o sobre los actuadores de propulsión. Su función es entregar al nivel superior una variable lógica de seguridad.

La interfaz principal es:

- `must_stop`

Cuando esta variable se activa, el nivel de control del vehículo debe anular la orden de avance o imponer una velocidad nula.

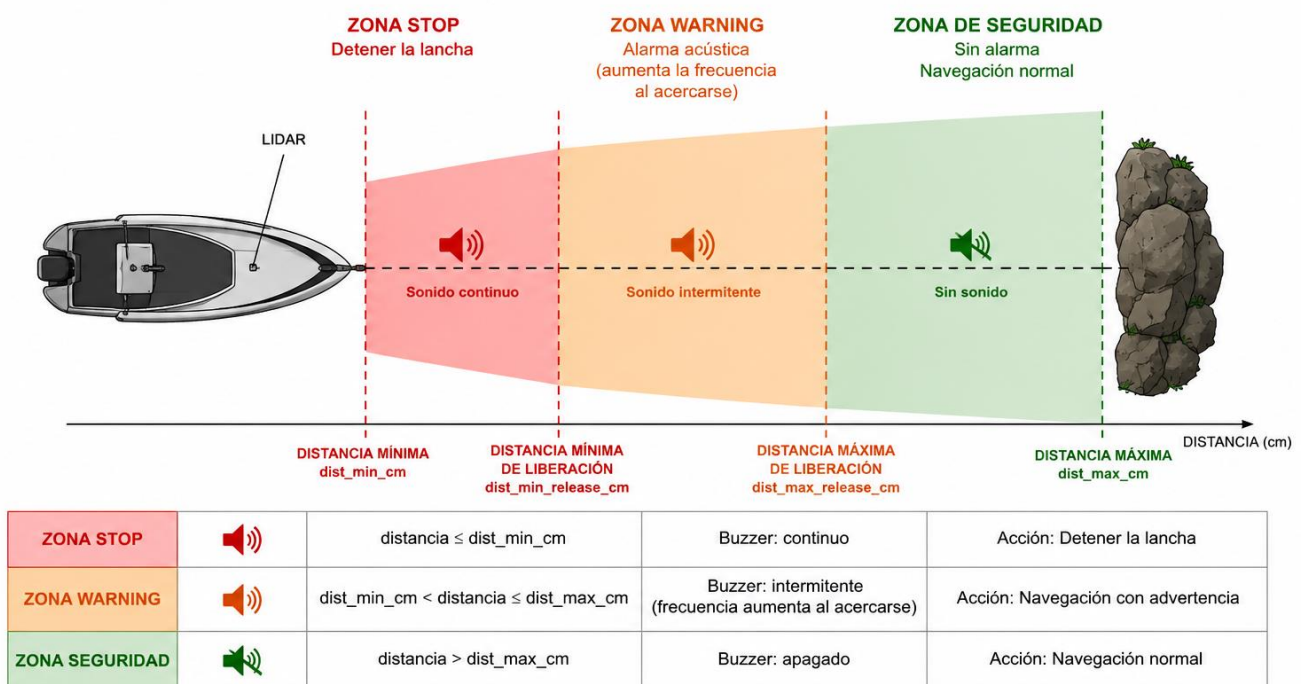
Ejemplo conceptual:

```
const LIDAR_ProximityState_t *lidar_state;
```

```
lidar_state = LIDAR_ProximityGetState();
```

```
if (lidar_state->must_stop != 0U)
{
    PWMControladorMotores(0, 0);
}
```

ZONAS DE PROXIMIDAD – LIDAR



HISTÉRESIS / LIBERACIÓN

- Para salir de ZONA STOP, la distancia debe ser $> dist_min_release_cm$
- Para salir de ZONA WARNING y volver a ZONA SEGURIDAD, la distancia debe ser $> dist_max_release_cm$

- Umbral de STOP
- Umbral de WARNING
- Umbral de SEGURIDAD

Resultado funcional esperado

Con la implementación actual, el subsistema LIDAR proporciona una primera capa de seguridad frontal basada en proximidad. Su operación puede resumirse del siguiente modo:

1. el sensor mide periódicamente la distancia al obstáculo;
2. la medición se valida y filtra;
3. la lógica clasifica la situación en zona segura, advertencia o parada;
4. el buzzer informa al operador sobre la proximidad del obstáculo;
5. si se alcanza la distancia mínima, el sistema genera una orden de parada.

De esta manera, el subsistema permite integrar una función básica de prevención de colisión, configurable desde telemetría y desacoplada del resto del sistema de navegación.